adding code coverage while juggling your RSpec tests with a batch of existing Minitest tests. I recommend adding a factory tool if there isn't one already in the mix. It's likely that writing tests for the legacy app will require creating complete chains of related objects. Setting up a factory tool to create the associations all at once saves a lot of time.

There are, of course, exceptions to this rule. Two spring to mind: a) the original developer has chosen a tool that's unsuitable for supporting the kind of testing weight you want to put on it or, more often, b) the existing tests are useless and it's best to delete them quickly and start over. At that point you can pick whatever tools you want.

Dependency Removal

Dependencies are the most challenging issue in legacy testing. Perhaps the greatest virtue of well-done TDD code is that the tests force individual pieces of the code to be maximally independent of each other.

Without tests, legacy code tends to be highly interdependent. This makes adding tests difficult in several ways. Multiple objects might need to be created to test a single method. It might be hard to limit a test to a true functional unit if that unit is hard to reach or encased in a massive 300-line method. There are ways to have the code you need to test be separate enough to enable the tests you must write.

Keep Things Separate

Perhaps the easiest way to keep your new code from being dependent on legacy code is to separate it yourself. Where possible, write new code in new methods or new classes, and merely call those from the existing legacy mess. In theory this leaves your new code unencumbered enough to be written via TDD.

Let's try a brief example. Consider a kind of messy method from a social-networking site called Flitter:

```
class Flit
  def process_flit
    if text =~ /##/
      flit.text = "testing: remove this code after 3/10/10"
    end
    if text.ends_with?("%fb%")
      send_to_facebook
    else if user.flits_in_last_day > 423
      return
    end
```